

SEC EDGAR Financial Intelligence RAG System

LangChain

LangGraph

FastAPI

Docker

Kubernetes

AWS EKS

CI/CD

RAG

Pinecone

STRATEGIC CONTEXT:

WHY THIS PROJECT WILL GET YOU HIRED: The SEC EDGAR Financial Intelligence RAG System is built on 20+ million real, publicly available government documents — no dummy data, no fake PDFs. Finance is the #1 sector hiring AI engineers right now. Every hedge fund, bank, and fintech is trying to build exactly this system. This project demonstrates LangChain, LangGraph, RAG, FastAPI, Docker, Kubernetes, AWS, and CI/CD — all the skills companies are filtering resumes for — in one real, enterprise-grade application. When a recruiter asks 'show me your work', you send one GitHub link and they see production architecture, not a tutorial copy.

PART A CERTIFICATIONS TO COMPLETE FIRST

Complete in this exact order | All available on Coursera | Auditable for free

01

IBM: Fundamentals of AI Agents Using RAG and LangChain

8 hours | Start Day 1

Platform: Coursera (IBM) | **Rating:** 4.8/5 | **Cost:** Free to audit

What you learn: RAG pipeline, LangChain components, FAISS vector search, building AI agents

Why first: Shortest course. Gets you to working RAG code in 8 hours. Foundation for everything else.

URL: coursera.org/learn/fundamentals-of-ai-agents-using-rag-and-langchain

02

IBM: Agentic AI with LangChain and LangGraph

3 weeks | Start after 01

Platform: Coursera (IBM) | **Rating:** 4.6/5 | **Cost:** Free to audit

What you learn: LangGraph stateful agents, multi-agent systems, agentic RAG, ReAct architecture

Why second: LangGraph appears in every senior AI job. This teaches it from IBM — recognized certification.

URL: coursera.org/learn/agentic-ai-with-langchain-and-langgraph

03

IBM: RAG and Agentic AI Professional Certificate (Full)

2-3 months | The main credential

Platform: Coursera (IBM) | **Rating:** 4.8/5 | **Cost:** ~\$49/month | Worth paying

What you learn: LangChain, LangGraph, CrewAI, AG2, MCP, RAG pipelines, vector stores, multimodal AI

Why this matters: IBM Professional Certificate on LinkedIn = instantly passes resume filters. Includes CrewAI and Model Context Protocol (MCP) — the newest frameworks companies are now requiring.

URL: coursera.org/professional-certificates/ibm-rag-and-agentic-ai

STUDY STRATEGY:

Start Course 01 today. While studying Course 01 and 02, simultaneously start building the project below. By the time you finish Course 03, your project will be ready to deploy. You learn and build at the same time.

PART B PROJECT: SEC EDGAR FINANCIAL INTELLIGENCE RAG SYSTEM

Enterprise-grade | Real data | All target skills in one system

PROJECT SUMMARY:

What it is: An enterprise AI analyst that lets users ask natural language questions about real SEC filings from US public companies (Apple, Tesla, Microsoft, etc.). Users ask: 'What are Apple's biggest risk factors in 2024?' or 'Compare Tesla and Ford revenue trends' — and the system retrieves the actual 10-K and 10-Q filings from SEC EDGAR, runs them through a LangGraph multi-agent pipeline, and returns accurate, cited answers. **Why it impresses:** This is the exact system hedge funds and investment banks are spending millions to build. You build it in 8-10 weeks and put it on GitHub.

DATA SOURCE — SEC EDGAR (100% Free, No License Required)

The US Securities and Exchange Commission (SEC) provides FREE public access to all financial filings from every public company since 1993 via their EDGAR system. No API key required for basic access. No copyright restrictions on the data. This is perfect — real, serious, enterprise-grade financial data that you can legally use in a public portfolio project.

Documents Available (FREE)

- › 10-K — Annual reports (key risk factors, financials)
- › 10-Q — Quarterly reports
- › 8-K — Material events (earnings, acquisitions)
- › S-1 — IPO prospectuses
- › DEF 14A — Proxy statements
- › Covers Apple, Tesla, Microsoft, Amazon, all S&P 500

Access Method

- › data.sec.gov — Free REST API, no auth needed
- › 10 requests/second rate limit (build in backoff)
- › Full text extraction of 10-K filings
- › Download PDF and HTML formats
- › XBRL financial data available as JSON
- › 20M+ filings since 1993 — unlimited corpus

COMPLETE TECH STACK

Python 3.11 LangChain v0.3 LangGraph FastAPI Pinecone OpenAI Embeddings Docker Kubernetes (AWS EKS)

GitHub Actions CI/CD AWS (EKS + ECR + S3) PostgreSQL (metadata) Redis (caching) LangSmith (observability)

Pydantic v2

Skills Demonstrated

- › LangChain (RAG pipeline)
- › LangGraph (multi-agent orchestration)
- › FastAPI (production REST API)
- › Pinecone (vector database)
- › Docker (containerisation)
- › Kubernetes on AWS EKS
- › GitHub Actions (CI/CD pipeline)
- › AWS ECR + S3 + EKS

What Companies See

- › RAG architecture from scratch
- › Stateful agent workflows
- › Production API design
- › Enterprise vector search
- › Containerised deployment
- › Cloud-native scaling
- › Automated testing + deploy
- › Full cloud deployment

PART C SYSTEM ARCHITECTURE

How the system works end-to-end

HIGH-LEVEL ARCHITECTURE (3 Layers)

LAYER 1 — DATA INGESTION PIPELINE

- › Python script fetches 10-K and 10-Q filings from SEC EDGAR API for chosen companies
- › PyMuPDF extracts clean text from HTML/PDF filings (you already know this from BidVeritas)
- › LangChain RecursiveCharacterTextSplitter chunks documents into 512-token overlapping segments
- › OpenAI text-embedding-3-large generates 3072-dimensional embeddings for each chunk
- › Pinecone stores vectors with metadata: company, filing_type, year, quarter, section, page_num
- › PostgreSQL stores filing metadata: company name, CIK, accession number, filing date
- › Ingestion runs on a schedule via GitHub Actions — new filings auto-ingested weekly

LAYER 2 — LANGGRAPH MULTI-AGENT QUERY PIPELINE

- › User sends natural language question via FastAPI endpoint
- › Router Agent (LangGraph node 1): classifies query type — single company, comparison, trend analysis
- › Retrieval Agent (LangGraph node 2): queries Pinecone with hybrid search (semantic + keyword BM25)
- › Analyst Agent (LangGraph node 3): Claude Sonnet or GPT-4o synthesises answer from retrieved chunks
- › Citation Agent (LangGraph node 4): attaches source references — company, filing type, page number
- › Guardrail Agent (LangGraph node 5): checks answer quality, hallucinates fallback if confidence low
- › State flows through LangGraph nodes with full checkpoint and memory — stateful multi-turn conversation

LAYER 3 — API + DEPLOYMENT LAYER

- › FastAPI application exposes: POST /query, POST /ingest, GET /companies, GET /health
- › Redis caches frequent queries — same question answered in <100ms after first call
- › Docker containerises the entire application with multi-stage build
- › Kubernetes (AWS EKS) handles auto-scaling — scales pods on query load
- › AWS ECR stores Docker images, AWS S3 stores raw filing documents as backup
- › GitHub Actions pipeline: on push to main — run tests → build Docker → push to ECR → deploy to EKS
- › LangSmith traces every LLM call for debugging and performance monitoring

PART D PHASE-BY-PHASE BUILD PLAN

8-10 weeks | Build while studying certifications simultaneously

HASE Data Pipeline + Basic RAG

Week 1-2

- › **Step 1:** Create Python virtual environment. Install: langchain, openai, pinecone-client, fastapi, pypdf, requests
- › **Step 2:** Write edgar_fetcher.py — fetches 10-K filings from data.sec.gov for 5 companies (Apple, Tesla, Microsoft, Amazon, Google). Use requests library with 1-second delays to respect SEC rate limit
- › **Step 3:** Write document_processor.py — extracts text from HTML filings using BeautifulSoup. Splits text into chunks with LangChain RecursiveCharacterTextSplitter (chunk_size=512, overlap=50)
- › **Step 4:** Create free Pinecone account (free tier: 1 index, 100K vectors — enough for testing). Create index with dimension=1536 (text-embedding-3-small) or 3072 (text-embedding-3-large)
- › **Step 5:** Write vector_store.py — embeds chunks with OpenAI embeddings and upserts to Pinecone with metadata dictionary
- › **Step 6:** Write basic_rag.py — takes a user question, embeds it, retrieves top-5 chunks from Pinecone, sends to GPT-4o with system prompt to answer only from context
- › **MILESTONE:** Ask 'What are Apple's risk factors?' in Python and get a cited answer from real SEC data

HASE FastAPI + LangChain Chains

Week 3-4

- › **Step 1:** Build FastAPI application (main.py). Create endpoint: POST /query that accepts {question: str, company: str}
- › **Step 2:** Replace basic RAG with LangChain LCEL chain: retriever | prompt | llm | output_parser
- › **Step 3:** Add LangChain ConversationBufferWindowMemory — system now remembers last 5 messages for follow-up questions
- › **Step 4:** Add GET /companies endpoint that returns list of ingested companies with filing counts
- › **Step 5:** Add POST /ingest endpoint that accepts {ticker: str} and triggers ingestion pipeline for that company
- › **Step 6:** Add Pydantic request and response models with full validation. Add HTTPException error handling
- › **Step 7:** Add Redis caching — cache query results for 1 hour to reduce API costs
- › **MILESTONE:** Working REST API with memory, caching, and proper error handling

HASE LangGraph Multi-Agent System

Week 5-6

- › **Step 1:** Install langgraph. Read LangGraph documentation for 2 hours before writing any code
- › **Step 2:** Define AgentState TypedDict: {messages, query_type, retrieved_docs, answer, citations, confidence}
- › **Step 3:** Build Router Node — uses GPT-4o-mini to classify query: single_company / comparison / trend. Returns updated state
- › **Step 4:** Build Retrieval Node — queries Pinecone differently based on query_type. Comparison queries retrieve from 2 companies simultaneously
- › **Step 5:** Build Analyst Node — synthesises answer from retrieved docs with company-specific system prompt
- › **Step 6:** Build Citation Node — extracts source metadata from retrieved docs and formats citation list
- › **Step 7:** Build Guardrail Node — checks if answer contains phrases like 'I don't know' or 'not in the documents'. If yes, routes back to Retrieval Node with expanded search
- › **Step 8:** Connect nodes with LangGraph edges. Add conditional edge: if confidence < 0.7, route back to retrieval
- › **Step 9:** Add LangSmith tracing with LANGCHAIN_API_KEY env variable — every agent run now traced
- › **MILESTONE:** Multi-agent pipeline handles comparison questions like 'Compare Apple and Microsoft revenue'

HASE Docker + GitHub Actions CI/CD

Week 7

- › **Step 1:** Install Docker Desktop. Write Dockerfile with multi-stage build: stage 1 installs dependencies, stage 2 copies only needed files
- › **Step 2:** Write docker-compose.yml with three services: api (FastAPI), redis, postgres. Test locally with docker-compose up
- › **Step 3:** Create GitHub repository. Add .gitignore for: .env, __pycache__, *.pyc, venv/
- › **Step 4:** Add GitHub Secrets: OPENAI_API_KEY, PINECONE_API_KEY, AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY
- › **Step 5:** Write .github/workflows/ci.yml — triggers on push to main. Steps: checkout code → run pytest → build Docker image → push to AWS ECR
- › **Step 6:** Write tests/test_api.py with pytest — test /health endpoint, test /query with a real question, test Pinecone connection
- › **MILESTONE:** Every git push automatically runs tests and builds a Docker image. Green checkmarks on GitHub

HASE AWS EKS Kubernetes Deployment

Week 8-9

- › **Step 1:** Create free AWS account. Install AWS CLI and kubectl. Configure AWS credentials
- › **Step 2:** Create AWS ECR repository (free for first 500MB). Push Docker image from Phase 4 to ECR
- › **Step 3:** Create EKS cluster using eksctl (smallest node group: 2x t3.small = ~\$30/month. Destroy after testing to save cost)

- › **Step 4:** Write `kubernetes/deployment.yaml` — defines Deployment with 2 replicas, resource limits, env vars from Kubernetes Secrets
- › **Step 5:** Write `kubernetes/service.yaml` — creates LoadBalancer service to expose FastAPI externally
- › **Step 6:** Write `kubernetes/hpa.yaml` — HorizontalPodAutoscaler scales pods from 2 to 10 based on CPU usage
- › **Step 7:** Update GitHub Actions workflow — add deploy step: `kubectl apply -f kubernetes/` after Docker push
- › **Step 8:** Screenshot the running pods with `kubectl get pods`. Save the AWS Console screenshot. These go in your README
- › **MILESTONE:** Application running on real Kubernetes cluster on AWS. Public URL serving queries

HASE GitHub README + Portfolio Polish

Week 10

- › **README.md must include:** Architecture diagram (draw with draw.io and export as PNG), live demo URL if possible, screenshots of the API working, tech stack badges, how to run locally in 3 commands
- › **HIDE from public GitHub:** Your OpenAI API key, Pinecone API key, AWS credentials (use `.env` file + `.gitignore`). Add a note: 'API keys required — contact for demo access'
- › **What TO show publicly:** All source code, Dockerfile, kubernetes YAML files, GitHub Actions workflow, tests, architecture diagram
- › **Add a DEMO.md:** Show 5 example questions and exact answers the system returns. With citations. This is what recruiters will read
- › **Add LangSmith trace screenshots:** Show the agent graph executing. This is something 99% of candidates cannot show

PART E EXACT FOLDER STRUCTURE FOR GITHUB

How to organise your repository to look professional

```

sec-rag-intelligence/
├── .github/
│   ├── workflows/
│   │   └── ci-cd.yml          # GitHub Actions pipeline
├── src/
│   ├── agents/
│   │   ├── router_agent.py    # LangGraph Router Node
│   │   ├── retrieval_agent.py # LangGraph Retrieval Node
│   │   ├── analyst_agent.py   # LangGraph Analyst Node
│   │   ├── citation_agent.py  # LangGraph Citation Node
│   │   └── guardrail_agent.py # LangGraph Guardrail Node
│   ├── pipeline/
│   │   ├── graph.py           # LangGraph StateGraph definition
│   │   └── state.py           # AgentState TypedDict
│   ├── ingestion/
│   │   ├── edgar_fetcher.py   # SEC EDGAR API client
│   │   └── doc_processor.py   # Text extraction + chunking
│   ├── vectorstore/
│   │   └── pinecone_store.py  # Pinecone upsert + retrieval
│   ├── api/
│   │   ├── main.py            # FastAPI application
│   │   ├── routes.py          # API route handlers
│   │   └── schemas.py         # Pydantic request/response models
│   ├── kubernetes/
│   │   ├── deployment.yaml
│   │   ├── service.yaml
│   │   └── hpa.yaml
│   ├── tests/
│   │   ├── test_api.py
│   │   ├── test_pipeline.py
│   │   └── test_retrieval.py
│   └── scripts/
│       └── ingest_companies.py # Bulk ingestion script
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
├── .env.example               # Template with empty keys
├── README.md                  # Architecture + setup guide
└── DEMO.md                    # 5 example queries with outputs

```

PART F WHAT TO HIDE vs SHOW ON GITHUB

Critical — do not expose secrets, do not hide your architecture

HIDE (add to .gitignore)

- > .env file (all API keys)
- > OPENAI_API_KEY value
- > PINECONE_API_KEY value
- > AWS_ACCESS_KEY_ID
- > AWS_SECRET_ACCESS_KEY
- > Any internal client notes
- > Your pricing / cost breakdown

SHOW (makes you look expert)

- > All .py source code files
- > Dockerfile + docker-compose.yml
- > kubernetes/ YAML files
- > .github/workflows/ci-cd.yml
- > tests/ directory with pytest
- > Architecture diagram PNG
- > .env.example (keys as empty strings)

PORTFOLIO STRATEGY:

The smart move: Add a README section called 'Live Demo'. Write: 'A live demo instance is available. Contact me at m.ahmad.aidigital@gmail.com with your use case for API access.' This sounds professional AND protects your API costs. Recruiters will email you — that is the goal.

PART G HOW TO DESCRIBE THIS PROJECT ON YOUR RESUME

Exact bullet points to copy — use engineering language, not marketing language

SEC EDGAR Financial Intelligence RAG System | github.com/YOUR_USERNAME/sec-rag-intelligence

- › Architected a production RAG system ingesting 10-K and 10-Q filings from SEC EDGAR for S&P; 500 companies — real financial corpus of 50,000+ document chunks stored in Pinecone with OpenAI text-embedding-3-large
- › Built a 5-node LangGraph stateful agent pipeline: Router → Retrieval → Analyst → Citation → Guardrail — handles single-company queries, cross-company comparisons, and multi-turn conversations with full checkpoint memory
- › Engineered FastAPI REST API with Redis query caching, Pydantic validation, and LangSmith observability tracing — p95 latency under 2 seconds for cached responses
- › Deployed to AWS EKS with Kubernetes HorizontalPodAutoscaler scaling from 2 to 10 replicas under load — Dockerised with multi-stage build, images stored in AWS ECR
- › Implemented GitHub Actions CI/CD pipeline — automated pytest suite, Docker build, ECR push, and kubectl deploy on every merge to main

PART H ESTIMATED COST TO BUILD THIS PROJECT

You do not need to spend much — strategic use of free tiers

Free Tier (Development)

- › SEC EDGAR API — 100% free forever
- › Pinecone free tier — 1 index, 100K vectors
- › OpenAI API — pay per use (~\$5 for testing)
- › GitHub Actions — 2,000 min/month free
- › AWS Free Tier — 12 months free (ECR, some EKS)
- › LangSmith — free tier available
- › Redis — run locally with Docker

Paid (Production, optional)

- › AWS EKS cluster: ~\$30/month (destroy after demo)
- › OpenAI embeddings: ~\$0.13 per 1M tokens
- › Pinecone Starter: \$70/month (use free to start)
- › Total for 2-month build: ~\$50-80 USD
- › Strategy: build + test locally, deploy to AWS
- › only for 1 week to take screenshots for portfolio
- › Then destroy cluster to stop billing

PART I COMPLETE TIMELINE SUMMARY

What to do every week — certification + project simultaneously

- Week 1** Start IBM Course 01 (RAG + LangChain). Build Phase 1: SEC fetcher + Pinecone ingestion
- Week 2** Finish Course 01. Complete Phase 1. Ask first question to real SEC data. Push to GitHub.
- Week 3** Start IBM Course 02 (LangGraph). Build Phase 2: FastAPI endpoints
- Week 4** Complete Phase 2. Working REST API. Start Course 03 (IBM Professional Certificate).
- Week 5-6** Build Phase 3: LangGraph 5-agent pipeline. Study Course 03 modules on LangGraph.

Week 7

Build Phase 4: Docker + GitHub Actions. CI/CD pipeline working. Green checkmarks.

Week 8-9

Build Phase 5: AWS EKS deployment. Take screenshots of live Kubernetes cluster.

Week 10

Phase 6: Polish README, write DEMO.md, update resume with project bullets.

Week 11+

Complete Course 03 certification. Add IBM Professional Certificate to LinkedIn. Apply.

PART J FINAL CHECKLIST — BEFORE YOU APPLY AGAIN

Do not apply until all of these are done

- › IBM Professional Certificate (RAG + Agentic AI) shows as credential on LinkedIn
- › GitHub repository is public with architecture diagram in README
- › At least 10 commits showing real development progress (not one giant commit)
- › DEMO.md shows 5 real questions and real answers from SEC data with citations
- › Resume updated with project bullet points using engineering language from Part G
- › Resume title updated to: AI ENGINEER | RAG & AGENTIC SYSTEMS
- › GitHub link visible at top of resume next to LinkedIn
- › Apply for: AI Engineer, AI Automation Engineer, LLM Engineer, RAG Engineer — not 'any AI role'
- › On Upwork: list SEC EDGAR RAG system as your primary project. Charge \$50-80/hr. Get first client.

FINAL MESSAGE:

You already know multi-agent design, Python, Supabase, vector embeddings, and production systems. You are not starting from zero. You are adding the industry-standard framework layer (LangChain, LangGraph) on top of skills you already have. This is 8-10 weeks of focused work — not a year. The SEC data is real. The system is real. The certification is real. After this, no recruiter can call you a no-code builder.